

Less-Numerical Algorithms

22.0 Introduction

You can stop reading now. You are done with *Numerical Recipes*, as such. This final chapter is an idiosyncratic set of “*less-numerical recipes*” that, for one reason or another, we have decided to include between the covers of an otherwise *more-numerically* oriented book. Authors of computer science texts, we’ve noticed, like to throw in a token numerical subject (usually quite a dull one — quadrature, for example). We find that we are not free of the reverse tendency.

Our selection of material is not completely arbitrary. In §9.0 we promised to provide a simple plotting routine. Another promised topic, Gray codes, was already used in the construction of quasi-random sequences (§7.8) and here needs only some additional explication. Two other topics, on diagnosing a computer’s floating-point parameters, and on arbitrary precision arithmetic, give additional insight into the machinery behind the casual assumption that computers are useful for doing things with real numbers (as opposed to integers or characters). The latter of these topics also shows a very different use for Chapter 12’s fast Fourier transform.

The three other topics (checksums, Huffman, and arithmetic coding) involve different aspects of data coding, compression, and validation. The material here is intended to be somewhat less abstract, and somewhat more practical, than the discussion of coding in §16.2, where coding was used to illustrate statistical aspects of state estimation. If you handle a large amount of data (numerical data, even), then a passing familiarity with these subjects might at some point come in handy. In §13.6, for example, we already encountered a good use for Huffman coding.

But again, you don’t have to read this chapter. (And you should learn about quadrature from Chapters 4 and 17, not from a computer science textbook!)

22.1 Plotting Simple Graphs

Yes, we all have our favorite plotting or graphics packages, and our favorite ways of generating plots from within C++ programs. But wait: Are your C++ pro-

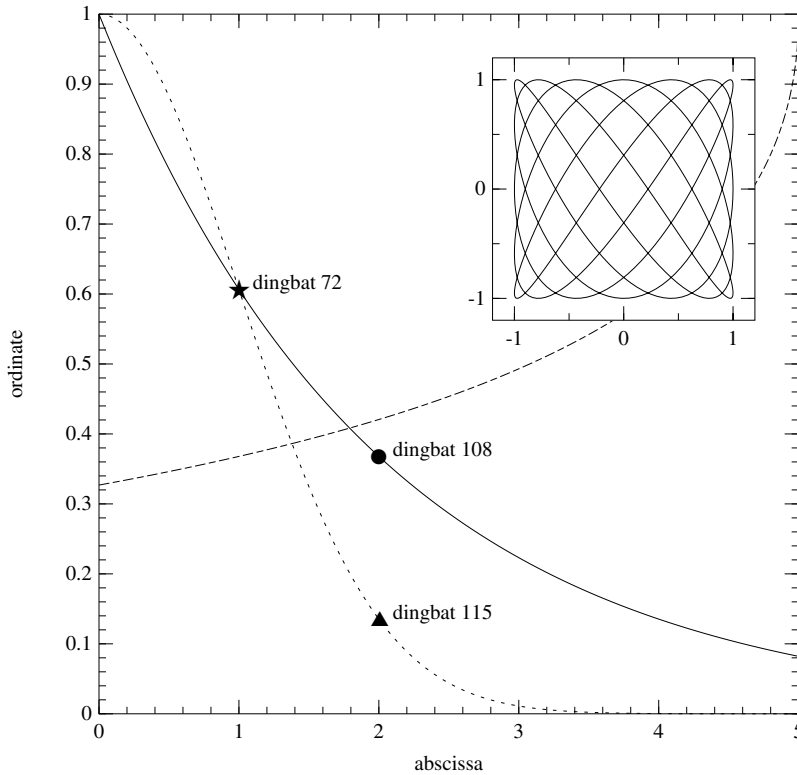


Figure 22.1.1. Simple plot generated using the objects `PSpace` and `PSpPlot`, which are wrappers for generating PostScript.

grams generating long text files of numbers, just so that you can read those numbers into a separate plotting or graphics package? If so, you might benefit from this section.

We find it useful to have on hand a couple of short C++ objects, implemented in simple source code, that generate simple plots. (By “simple,” we mean “like most of the figures in this book.”) We are then able to make plots from anywhere in our programs, whether as an aid in debugging or as final output. Equally important, we can make changes to the plotting source code at will, adding features or modifying the look of the plot.

One way of accomplishing these goals is by means of a C++ “wrapper” that does no more, nor less, than to write out a valid PostScript [1] file, which can be viewed or printed using a PostScript viewer such as the freely available Ghostscript/GSview [2]. In fact, the viewer can readily be invoked by a method within the wrapper object, so that the plot simply pops up in its own window on your screen.

An example will make this clearer. Figure 22.1.1 shows a sample plot that has a couple of x , y scaled boxes, some lines and points of varying types, and some text labels. Here is the code that generates the figure:

```
void pspplot_example() {
    Routine for creating Figure 22.1.1.
    VecDoub x1(500),x2(500),y1(500),y2(500),y3(500),y4(500);
    for (Int i=0;i<500;i++) {
        Generate some data.
```

[psplotexample.h](#)

```

    x1[i] = 5.*i/499.;
    y1[i] = exp(-0.5*x1[i]);
    y2[i] = exp(-0.5*SQR(x1[i]));
    y3[i] = exp(-0.5*sqrt(5.-x1[i]));
    x2[i] = cos(0.062957*i);
    y4[i] = sin(0.088141*i);
}

PSPage pg("d:\\nr3\\newchap20\\myplot.ps");
PSPlot plot1(pg,100.,500.,100.,500.);
plot1.setlimits(0.,5.,0.,1.);
plot1.frame();
plot1.autoscales();
plot1.xlabel("abscissa");
plot1.ylabel("ordinate");
plot1.lineplot(x1,y1);
plot1.setdash("2 4");
plot1.lineplot(x1,y2);
plot1.setdash("6 2 4 2");
plot1.lineplot(x1,y3);
plot1.setdash("");
plot1.pointsymbol(1.,exp(-0.5),72,16.);
plot1.pointsymbol(2.,exp(-1.),108,12.);
plot1.pointsymbol(2.,exp(-2.),115,12.);
plot1.label("dingbat 72",1.1,exp(-0.5));
plot1.label("dingbat 108",2.1,exp(-1.));
plot1.label("dingbat 115",2.1,exp(-2.));

PSPlot plot2(pg,325.,475.,325.,475.);
plot2.clear();
plot2.setlimits(-1.2,1.2,-1.2,1.2);
plot2.frame();
plot2.scales(1.,0.5,1.,0.5);
plot2.lineplot(x2,y4);

pg.close();
pg.display();
}

```

Instantiate a page.

Instantiate a plot on the page. Position is specified in pt (72 pt = 1 in, or 28 pt = 1 cm).

Unsets dash.

Instantiate a second plot.

Erase what's underneath it.

Pop up a window displaying the plot file.

The general idea is that a `PSPage` object (`pg` in the example above) represents a whole sheet of paper, or window on the screen. It can contain one or more `PSPlot` objects. In the above example there are two: `plot1` and `plot2`. `PSPlot` objects can be separate on the page, or overlapping. Each has its own x, y coordinate system, its own x - and y -axis labels, and so forth. With no more explanation than this, you should be able to find a program line above that corresponds to each feature in the figure. The last line makes the plot pop up on our screen.

Point symbols are referenced by their character number in the Zapf Dingbats font, which is built into PostScript. If you want to see all the possible symbols, a Web search for “LaTeX Postscript Dingbats” will turn up several charts. Or, just write a program to plot them all. (Hint: There are possibly useful symbols from 33 to 126, and from 161 to 254.)

A Webnote [3] gives the complete source code for the `PSPage` and `PSPlot` objects, which is only about 150 lines long. In the course of writing this book, our personal version of the code expanded to about 450 lines. This is an order of magnitude or two less than the standard packages that are available in open source code, `GNUPLOT`, for example [4]. It is a question of trading off capability (theirs much greater) for ease of modifying the source code (you be the judge).